

# 协议数据的捕获和解析

## 实验二 • 实验报告

姓 名 张恒基  
学 号 2024210926  
班 级 2024211301  
完成日期 2026 年 6 月

北京邮电大学

Beijing University of Posts and Telecommunications

## 摘要

---

本报告围绕北京邮电大学计算机网络实验二「协议数据的捕获和解析」，在 Windows 11 校园网环境下使用 Wireshark 对 IP、ICMP、DHCP、ARP、TCP 五类典型协议进行抓包与字段级分析。实验依次完成 ICMP ping（含大包分片）、DHCP 地址申请（DORA）、ARP 地址解析以及 TCP 连接建立、数据传输与释放全过程的捕获；在 Wireshark 中展开协议树，抄录首部字段，验证 IPv4 首部校验和算法，分析 IP 分片标识与偏移关系，对比 ICMP Echo 请求/应答、ARP 广播/单播差异，并追踪 TCP 三次握手、确认应答与四次挥手中的序号规律。

全文按指导书第 9 节结构组织，所有字段值均从 `captures/` 下 `.pcapng` 经 tshark 解析抄录，并附字段摘录图与 Mermaid 时序图。

关键词：Wireshark • IP 分片 • ICMP • DHCP • ARP • TCP 握手

# 目录

---

|                            |    |
|----------------------------|----|
| 摘要 .....                   | 2  |
| 1 实验目的 .....               | 1  |
| 2 实验环境 .....               | 2  |
| 3 实验内容和实验步骤 .....          | 3  |
| 3.1 实验任务 .....             | 3  |
| 3.2 ICMP 数据捕获 .....        | 3  |
| 3.3 DHCP 数据捕获 .....        | 3  |
| 3.4 ARP 数据捕获 .....         | 3  |
| 3.5 TCP 数据捕获 .....         | 4  |
| 4 IP 协议分析 .....            | 5  |
| 4.1 IP 首部字段 .....          | 5  |
| 4.2 IP 首部校验和验证 .....       | 6  |
| 4.3 IP 分片分析 .....          | 7  |
| 5 ICMP 协议分析 .....          | 9  |
| 6 DHCP 协议分析 .....          | 11 |
| 6.1 DHCP 报文过程 (DORA) ..... | 11 |
| 6.2 DHCP 字段和配置参数 .....     | 12 |
| 7 ARP 协议分析 .....           | 14 |
| 8 TCP 协议分析 .....           | 16 |
| 8.1 TCP 首部字段 .....         | 16 |
| 8.2 三次握手 .....             | 16 |
| 8.3 数据传输 .....             | 17 |
| 8.4 四次挥手 .....             | 18 |
| 9 实验中遇到的问题和解决方案 .....      | 20 |
| 10 实验心得 .....              | 21 |

## 实验目的

---

通过本实验，我希望达成以下目标：

1. 掌握网络协议分析仪的使用方法：熟悉 Wireshark 的捕获过滤器、显示过滤器，以及 Packet List / Packet Detail / Packet Bytes 三栏的协同分析方式。
2. 理解 IP 层数据报结构：能够读懂 IPv4 首部各字段含义，手工验证首部校验和，并解释大包 ping 触发的 IP 分片机制。
3. 认识常见网络控制协议：分析 ICMP 诊断报文、DHCP 动态配置流程、ARP 地址解析过程，建立「协议字段 ↔ 网络行为」的对应关系。
4. 深入理解 TCP 可靠传输：从真实抓包中观察三次握手、序号确认、窗口与 MSS，以及连接释放的四次挥手过程。

# 实验环境

---

|        |                                        |
|--------|----------------------------------------|
| 操作系统   | Windows 11                             |
| 抓包工具   | Wireshark 4.6.6 + Npcap                |
| 网络环境   | 校园 Wi-Fi (WLAN)                        |
| 本机 IP  | 10.122.219.108                         |
| 子网掩码   | 255.255.192.0                          |
| 默认网关   | 10.122.192.1                           |
| 本机 MAC | c0:35:32:78:d3:09 (以 ipconfig /all 为准) |

表 1 实验软硬件环境

## 实验内容和实验步骤

---

### 实验任务

本实验需完成以下五项协议分析任务：

1. IP 协议：首部字段表、首部校验和验证、分片分析 ICMP 协议：Echo Request / Reply 字段对比 DHCP 协议：DORA 四阶段与配置参数提取 ARP 协议：Request / Reply 字段与广播/单播分析 TCP 协议：首部字段、三次握手、数据传输、四次挥手

### ICMP 数据捕获

#### 过滤器与命令

- Capture Filter: `icmp`
- Display Filter: `icmp`
- 普通 ping: `ping www.bupt.edu.cn`
- 大包分片: `ping -l 8000 10.122.192.1`

操作步骤：在 WLAN 网卡开始捕获 → 执行 ping 命令 → 停止捕获 → 分别保存为 `captures/icmp/icmp-normal.pcapng` 与 `captures/icmp/icmp-large-packet-fragmentation.pcapng`。

### DHCP 数据捕获

#### 过滤器与命令

- Capture Filter: `udp port 67`
- Display Filter: `bootp`
- 释放 IP: `ipconfig /release`
- 重新申请: `ipconfig /renew`（需管理员权限）

保存为 `captures/dhcp/dhcp-dora.pcapng`，验证 Transaction ID 相同的 Discover / Offer / Request / ACK 四包齐全。

### ARP 数据捕获

#### 过滤器与命令

- Capture Filter: `arp`
- Display Filter: `arp`
- 清空缓存: `arp -d *`
- 触发解析: `ping 10.122.192.1`

保存为 `captures/arp/arp-request-reply.pcapng` 。

## TCP 数据捕获

### 过滤器与命令

- Capture Filter: `tcp port 80`
- Display Filter: `tcp.port == 80`
- 触发流量: `curl -4 http://example.com`

保存为 `captures/tcp/tcp-http.pcapng`，确认含 SYN 握手、数据段与 FIN 挥手（`tcp.stream==0`）。

# IP 协议分析

抓包文件：`captures/icmp/icmp-normal.pcapng`（帧 1）、`captures/icmp/icmp-large-packet-fragmentation.pcapng`（分片 Identification `0xCFBA`）。

## IP 首部字段

选定帧 1：本机 `10.122.219.108` → `10.3.19.2`（ping `www.bupt.edu.cn`）的 ICMP Echo Request 承载 IPv4 数据报。

| 字段                      | 捕获值            | 长度     | 功能说明          |
|-------------------------|----------------|--------|---------------|
| Version                 | 4              | 4 bit  | IPv4          |
| Header Length           | 20 bytes (5)   | 4 bit  | 首部长度，单位 4 字节  |
| Differentiated Services | 0x00           | 8 bit  | DSCP/ECN 均为 0 |
| Total Length            | 60             | 16 bit | IP 数据报总长度（字节） |
| Identification          | 0xCFB5         | 16 bit | 分片标识          |
| Flags                   | DF=0, MF=0     | 3 bit  | 未分片           |
| Fragment Offset         | 0              | 13 bit | 分片偏移（×8 字节）   |
| Time to Live            | 128            | 8 bit  | 生存时间 TTL      |
| Protocol                | ICMP (1)       | 8 bit  | 上层协议类型        |
| Header Checksum         | 0x6820         | 16 bit | 首部校验和         |
| Source Address          | 10.122.219.108 | 32 bit | 源 IP          |
| Destination Address     | 10.3.19.2      | 32 bit | 目的 IP         |

表 2 IPv4 首部字段（帧 1）



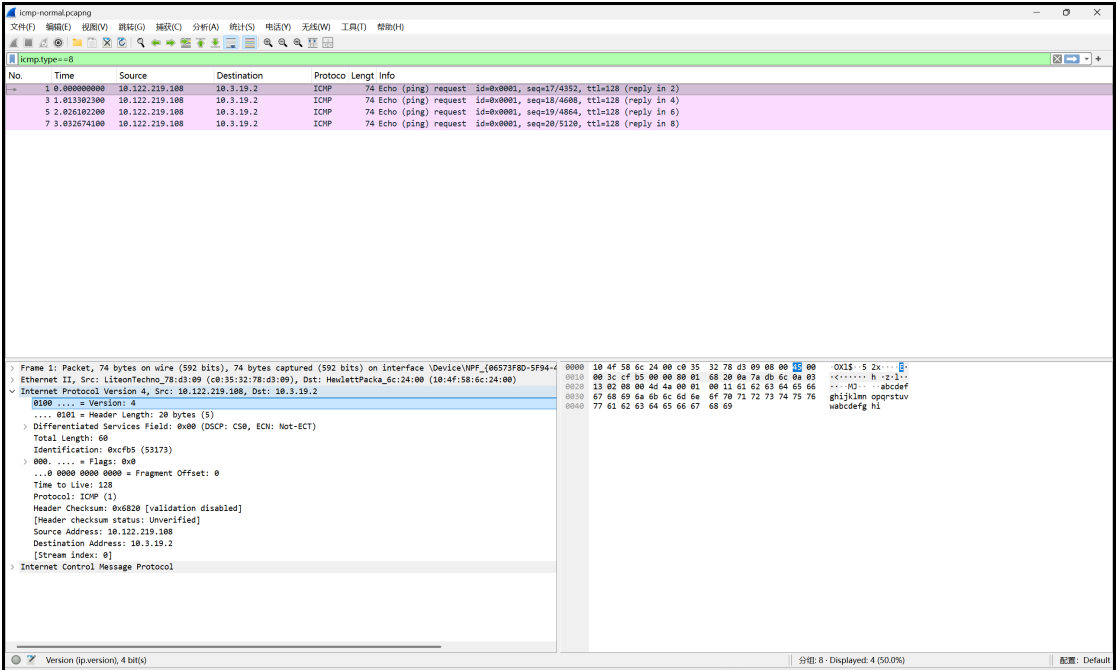


图 1 IPv4 首部字段 Wireshark 截图（见图）

Packet Bytes 首部十六进制（20 字节）：

```
45 00 00 3c cf b5 00 00 80 01 68 20 0a 7a db 6c 0a 03 13 02
```

IP 首部校验和验证

IPv4 首部校验和采用 16 位反码求和（one’ s complement sum）：

- 1. 将校验和字段（第 11 - 12 字节）置 0；
- 2. 将首部按 16 bit 分组求和，溢出高位回卷到低位；
- 3. 对结果按位取反，即为校验和。

验证过程（帧 1）

- 1. 复制 20 字节首部： 4500 003c cfb5 0000 8001 6820 0a7a db6c 0a03 1302
- 2. 校验和置零： 4500 003c cfb5 0000 8001 0000 0a7a db6c 0a03 1302
- 3. 按 16 bit 分成 10 组：

| 组号 | 十六进制   | 十进制   |
|----|--------|-------|
| 1  | 0x4500 | 17664 |
| 2  | 0x003C | 60    |
| 3  | 0xCFB5 | 53045 |
| 4  | 0x0000 | 0     |
| 5  | 0x8001 | 32769 |
| 6  | 0x0000 | 0     |

| 组号 | 十六进制   | 十进制   |
|----|--------|-------|
| 7  | 0x0A7A | 2682  |
| 8  | 0xDB6C | 56172 |
| 9  | 0x0A03 | 2563  |
| 10 | 0x1302 | 4866  |

表 3 IP 首部 16 bit 分组

#### 4. 反码相加（含回卷）：

```

0x4500 + 0x003C = 0x453C
0x453C + 0xCFB5 = 0x114F1 → 回卷 → 0x14F2
0x14F2 + 0x8001 = 0x94F3
0x94F3 + 0x0A7A = 0x9F6D
0x9F6D + 0xDB6C = 0x17AD9 → 回卷 → 0x7ADA
0x7ADA + 0x0A03 = 0x84DD
0x84DD + 0x1302 = 0x97DF

```

#### 5. 取反：~0x97DF = 0x6820

#### 6. 与 Wireshark 对比：显示 0x6820，一致 ✓

scripts/verify\_ipv4\_checksum.py 交叉验证输出同为 0x6820。

## IP 分片分析

ping -l 8000 抓包中 Identification = 0xCFBA 的 6 个出站分片（帧 13 - 18）：

| 分片序号 | Identification | MF | Fragment Offset | Total Length |
|------|----------------|----|-----------------|--------------|
| 1    | 0xCFBA         | 1  | 0               | 1500         |
| 2    | 0xCFBA         | 1  | 185             | 1500         |
| 3    | 0xCFBA         | 1  | 370             | 1500         |
| 4    | 0xCFBA         | 1  | 555             | 1500         |
| 5    | 0xCFBA         | 1  | 740             | 1500         |
| 6    | 0xCFBA         | 0  | 925             | 628          |

表 4 同一原始数据报的分片对比

分析说明：

- Identification 均为 0xCFBA，表明 6 个分片属于同一原始 IP 数据报。
- 前 5 个分片 MF=1，最后一个 MF=0，表示分片结束。

- Fragment Offset 递增:  $0 \rightarrow 185 \rightarrow 370 \rightarrow \dots \rightarrow 925$  (单位 8 字节), 接收端按偏移重组。
- 除末片 628 字节外, 其余各片 1500 字节 (接近以太网 MTU)。

原始 IP 数据报 (> MTU)

|          |          |          |          |          |         |
|----------|----------|----------|----------|----------|---------|
|          |          |          |          |          |         |
| 分片1      | 分片2      | 分片3      | 分片4      | 分片5      | 分片6     |
| off=0    | off=185  | off=370  | off=555  | off=740  | off=925 |
| MF=1     | MF=1     | MF=1     | MF=1     | MF=1     | MF=0    |
| len=1500 | len=1500 | len=1500 | len=1500 | len=1500 | len=628 |
|          |          |          |          |          |         |

Identification = 0xCFBA (全部相同)

图 1 IP 分片示意图

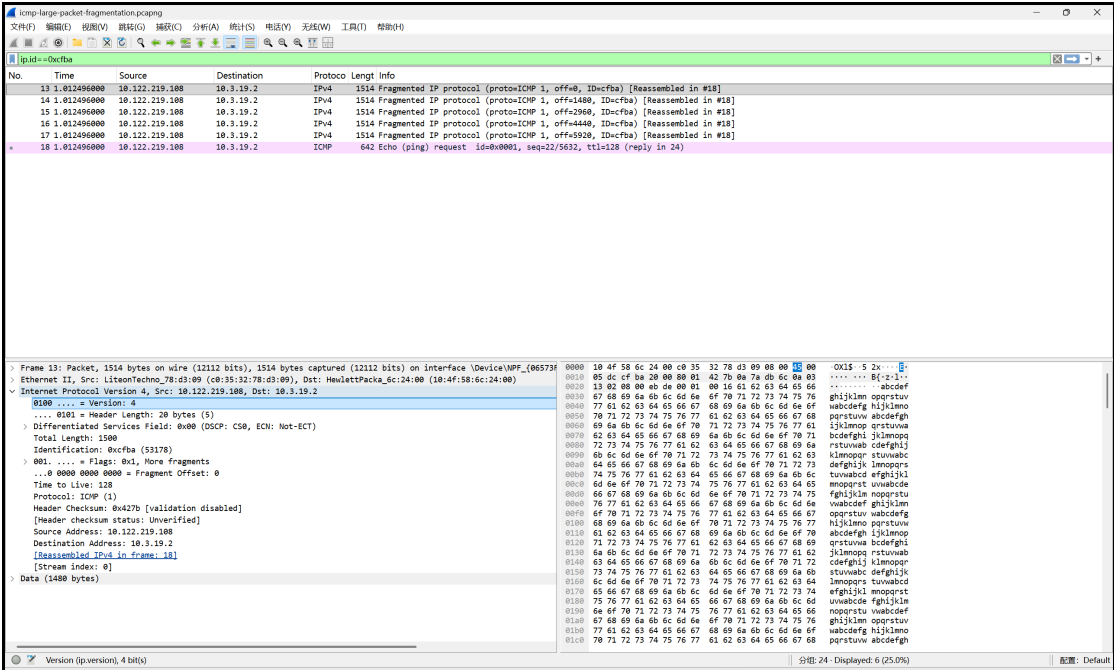


图 2 IP 分片对比 Wireshark 截图 (见图)

# ICMP 协议分析

抓包文件：`captures/icmp/icmp-normal.pcapng`（`ping www.bupt.edu.cn`）。

ICMP（Internet Control Message Protocol）是网络层控制报文协议，用于差错报告与网络诊断。`ping` 使用 Echo Request（Type=8）与 Echo Reply（Type=0）检测连通性。

选定帧 1（Request）与帧 2（Reply），序号均为 17：

| 字段              | Request                          | Reply                            | 功能        |
|-----------------|----------------------------------|----------------------------------|-----------|
| Type            | 8                                | 0                                | 报文类型      |
| Code            | 0                                | 0                                | 类型细分      |
| Checksum        | 0x4D4A                           | 0x554A                           | ICMP 校验和  |
| Identifier      | 1                                | 1                                | ping 会话标识 |
| Sequence Number | 17                               | 17                               | 序号        |
| Data            | abcdefghijklmnopqrstuvwabcdefghi | abcdefghijklmnopqrstuvwabcdefghi | 32 字节测试数据 |

表 5 ICMP Echo 请求与应答字段对比（帧 1 & 2）

会话关联说明：

- Identifier 相同（均为 1），表明属于同一次 `ping` 进程。
- Sequence Number 对应（均为 17），表明为第 17 号请求的应答。
- 源/目的 IP 互换：Request `10.122.219.108` → `10.3.19.2`，Reply `10.3.19.2` → `10.122.219.108`。
- Reply 的 IP TTL 为 58，Request TTL 为 128（Windows 默认）。

结论：Identifier + Sequence Number 双字段匹配，可唯一确定 Request/Reply 配对关系。

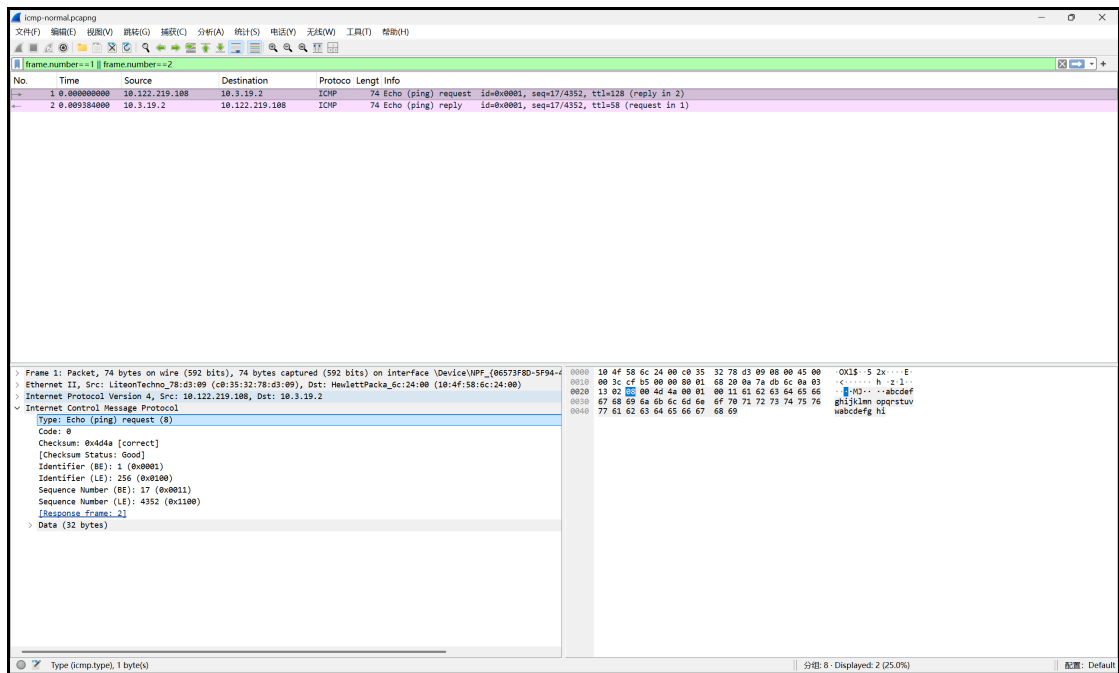


图 3 ICMP Echo Request / Reply Wireshark 截图（见图）

# DHCP 协议分析

抓包文件：`captures/dhcp/dhcp-dora.pcapng`（`ipconfig /release` + `ipconfig /renew`）。

DHCP（Dynamic Host Configuration Protocol）动态主机配置协议，客户端无需手工配置即可从服务器获取 IP 地址、子网掩码、默认网关、DNS 服务器及租约时间等参数。

## DHCP 报文过程（DORA）

Transaction ID 均为 `0x348015F3` 的四个包（帧 2 - 5）：

| 阶段 | 报文类型          | 源地址      | 目的地址            | 主要作用       |
|----|---------------|----------|-----------------|------------|
| 1  | DHCP Discover | 0.0.0.0  | 255.255.255.255 | 客户端广播寻找服务器 |
| 2  | DHCP Offer    | 10.3.9.2 | 10.122.219.108  | 服务器单播提供 IP |
| 3  | DHCP Request  | 0.0.0.0  | 255.255.255.255 | 客户端广播请求使用  |
| 4  | DHCP ACK      | 10.3.9.2 | 10.122.219.108  | 服务器单播确认分配  |

表 6 DHCP DORA 四阶段

帧 1 为 DHCP Release (Transaction ID `0x9F16ECB3`)，属于 `/release` 操作，不属于本次 DORA 流程。

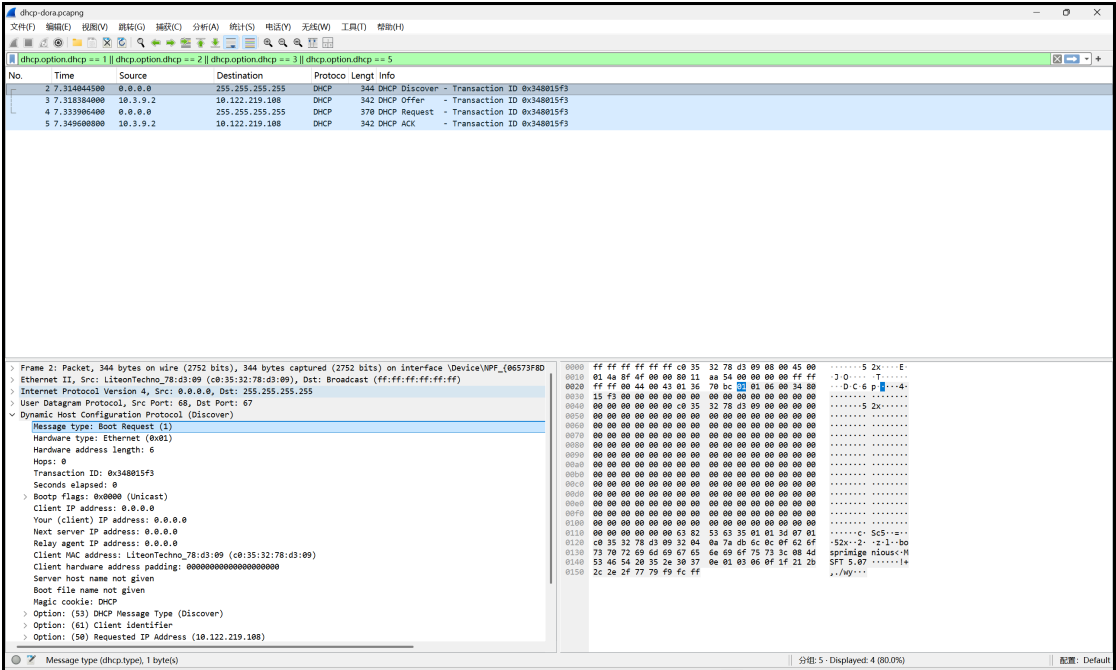


图 4 DHCP DORA 四包 Wireshark 截图（见图）

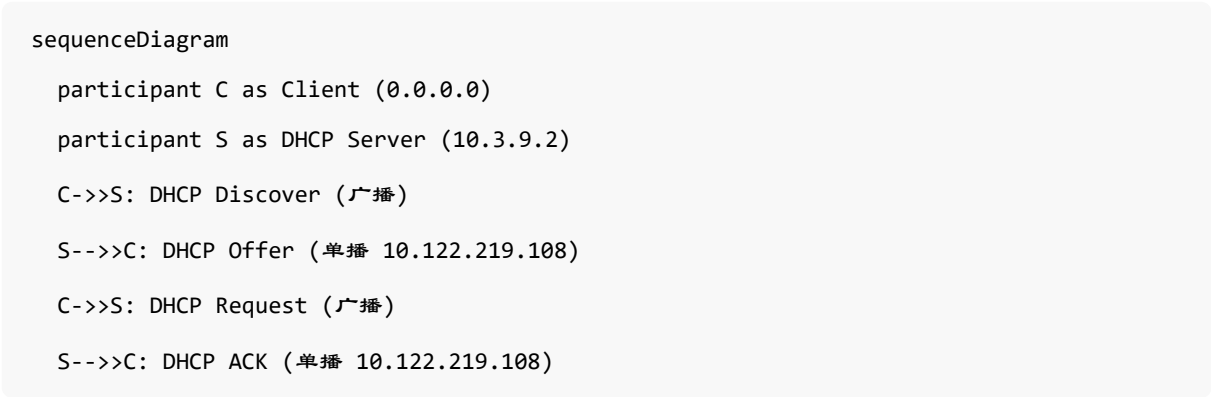


图 2 DHCP DORA 消息序列图

## DHCP 字段和配置参数

从 DHCP ACK（帧 5）提取：

| 参数                 | Option | 捕获值                                | 含义         |
|--------------------|--------|------------------------------------|------------|
| Your IP Address    | —      | 10.122.219.108                     | 分配给客户端的 IP |
| Subnet Mask        | 1      | 255.255.192.0                      | 子网掩码       |
| Router             | 3      | 10.122.192.1                       | 默认网关       |
| Domain Name Server | 6      | 10.3.9.5,<br>10.3.9.4,<br>10.3.9.6 | DNS 服务器    |

| 参数                        | Option | 捕获值         | 含义         |
|---------------------------|--------|-------------|------------|
| IP Address<br>Lease Time  | 51     | 3600 (1 小时) | 租约时间       |
| DHCP Server<br>Identifier | 54     | 10.3.9.2    | DHCP 服务器标识 |

表 7 DHCP ACK 配置参数

服务器角色判断:

- DHCP 服务器为 10.3.9.2 (Option 54), 不是默认网关 10.122.192.1。
- 网关 10.122.192.1 仅作为 Option 3 (Router) 下发。
- giaddr 均为 0.0.0.0, 客户端与服务器在同一广播域, 无 DHCP Relay。



# ARP 协议分析

抓包文件：`captures/arp/arp-request-reply.pcapng`（`ping 10.122.192.1` 触发）。

ARP（Address Resolution Protocol）根据 IP 地址解析 MAC 地址，使以太网帧能正确封装目的物理地址。

| 字段            | Request           | Reply             | 功能       |
|---------------|-------------------|-------------------|----------|
| Hardware Type | 1                 | 1                 | Ethernet |
| Protocol Type | 0x0800            | 0x0800            | IPv4     |
| Hardware Size | 6                 | 6                 | MAC 6 字节 |
| Protocol Size | 4                 | 4                 | IP 4 字节  |
| Opcode        | 1                 | 2                 | 请求 / 应答  |
| Sender MAC    | 10:4f:58:6c:24:00 | c0:35:32:78:d3:09 | 发送方 MAC  |
| Sender IP     | 10.122.192.1      | 10.122.219.108    | 发送方 IP   |
| Target MAC    | 00:00:00:00:00:00 | 10:4f:58:6c:24:00 | 目标 MAC   |
| Target IP     | 10.122.219.108    | 10.122.192.1      | 目标 IP    |

表 8 ARP Request / Reply 字段对比

广播与单播：

- ARP Request（帧 1）：Opcode=1，Target MAC 为 `00:00:00:00:00:00`。网关 `10.122.192.1` 向本机发起询问，以太网目的地址为本机 MAC `c0:35:32:78:d3:09`（单播 ARP）。
- ARP Reply（帧 2）：Opcode=2，本机以 `c0:35:32:78:d3:09` 单播回复网关 MAC `10:4f:58:6c:24:00`。

核心原理：用广播（或单播）询问谁拥有某 IP，目标主机单播返回自己的 MAC。

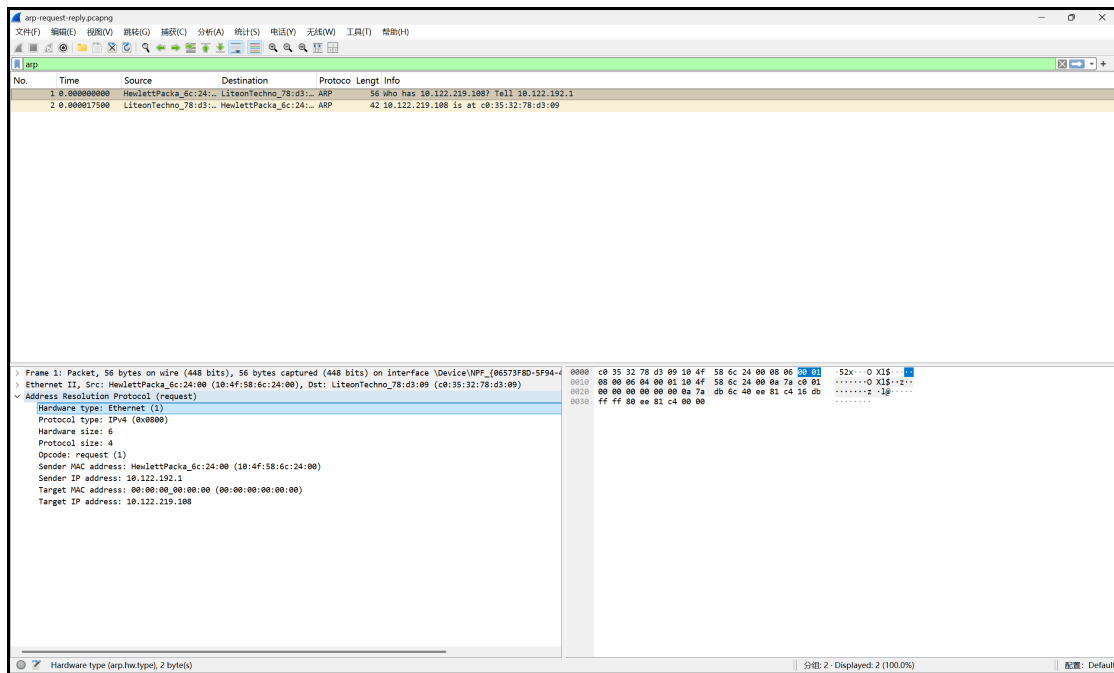


图 5 ARP Request / Reply Wireshark 截图（见图）

# TCP 协议分析

抓包文件：`captures/tcp/tcp-http.pcapng`（`curl -4 http://example.com`，`10.122.219.108:60247 ↔ 172.66.147.243:80`）。

## TCP 首部字段

以帧 1（客户端 SYN，`60247 → 80`，`http://example.com`）为例：

| 字段                    | 长度     | 捕获示例      | 功能       |
|-----------------------|--------|-----------|----------|
| Source Port           | 16 bit | 60247     | 源端口      |
| Destination Port      | 16 bit | 80        | HTTP 端口  |
| Sequence Number       | 32 bit | 524890319 | 初始序号 ISN |
| Acknowledgment Number | 32 bit | 0         | SYN 无确认号 |
| Header Length         | 4 bit  | 32 bytes  | 含 MSS 选项 |
| Flags                 | 若干位    | SYN       | 请求连接     |
| Window Size           | 16 bit | 65535     | 接收窗口     |
| Urgent Pointer        | 16 bit | 0         | 无紧急数据    |
| Options (MSS)         | 可变     | 1460      | 最大段长度    |

表 9 TCP 首部字段（帧 1）

## 三次握手

连接 `10.122.219.108:60247 ↔ 172.66.147.243:80`（帧 1 - 3）：

| 步骤 | 方向            | Flags    | Seq        | Ack        | 说明            |
|----|---------------|----------|------------|------------|---------------|
| 1  | Client→Server | SYN      | 524890319  | —          | 客户端请求连接       |
| 2  | Server→Client | SYN, ACK | 4130144482 | 524890320  | Ack = Seq+1 ✓ |
| 3  | Client→Server | ACK      | 524890320  | 4130144483 | Ack = Seq+1 ✓ |

表 10 TCP 三次握手

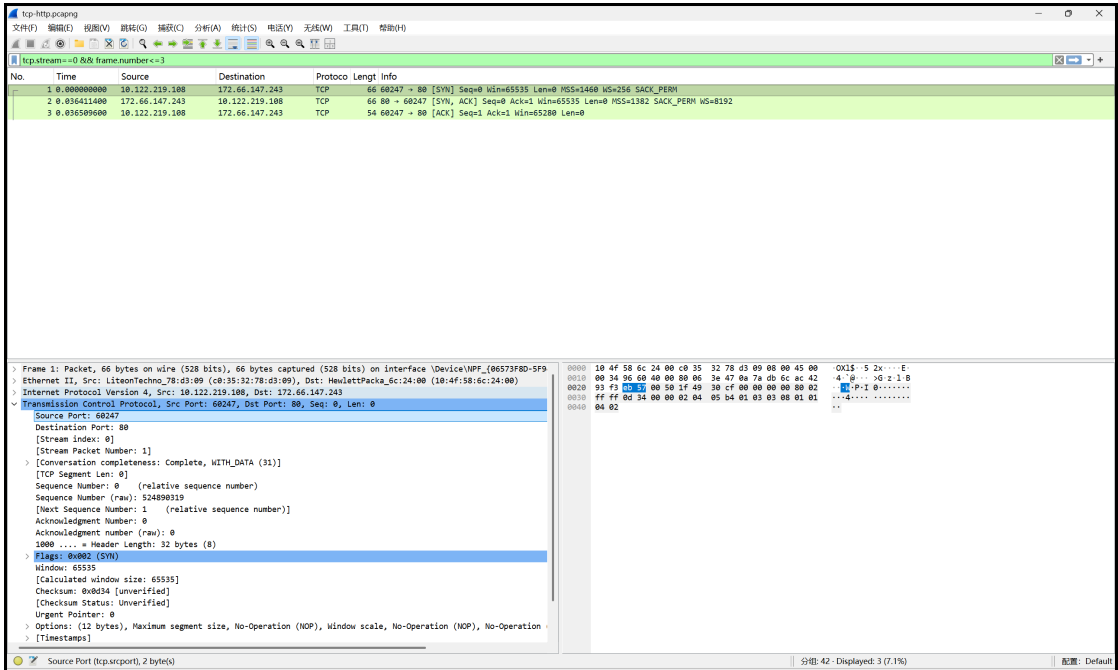


图 6 TCP 三次握手 Wireshark 截图

sequenceDiagram

participant C as Client

participant S as Server

C->>S: SYN Seq=524890319

S->>C: SYN+ACK Seq=4130144482 Ack=524890320

C->>S: ACK Seq=524890320 Ack=4130144483

图 3 TCP 三次握手时序图

数据传输

帧 4 客户端发送 Seq=524890320, Len=75 ； 帧 5 服务器 Ack=524890395 。

验证： 524890395 = 524890320 + 75 ✓

1. Window Size (帧 5)：16 字节。
2. MSS：客户端 1460，服务器 1382。

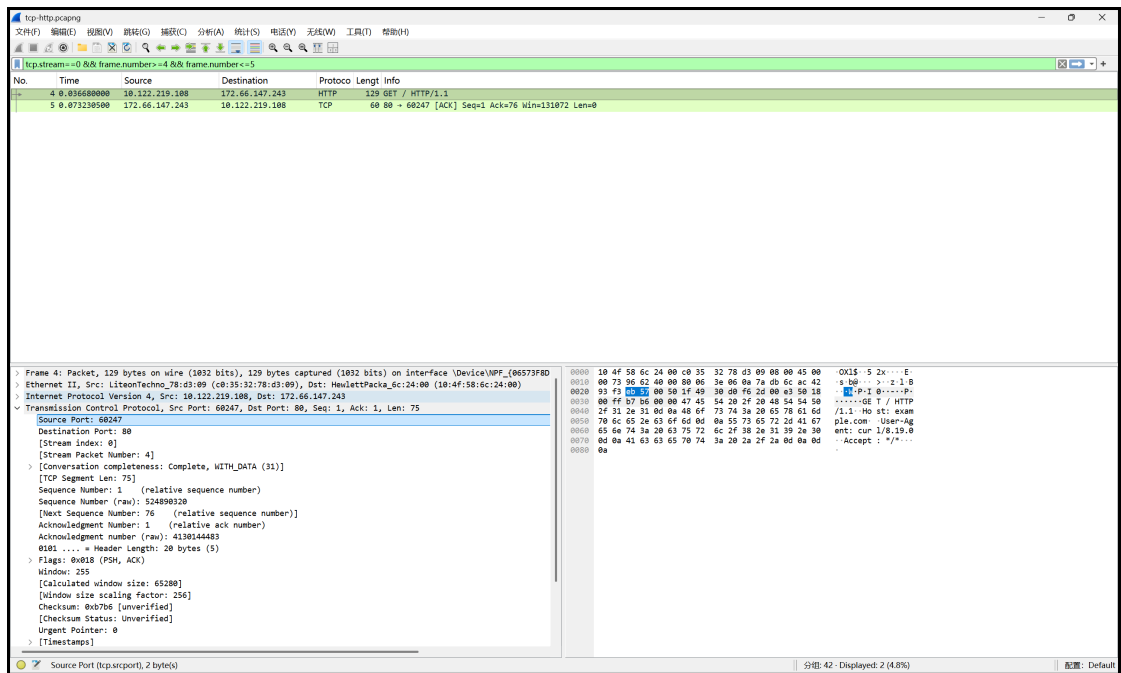


图 7 TCP 数据传输 Wireshark 截图

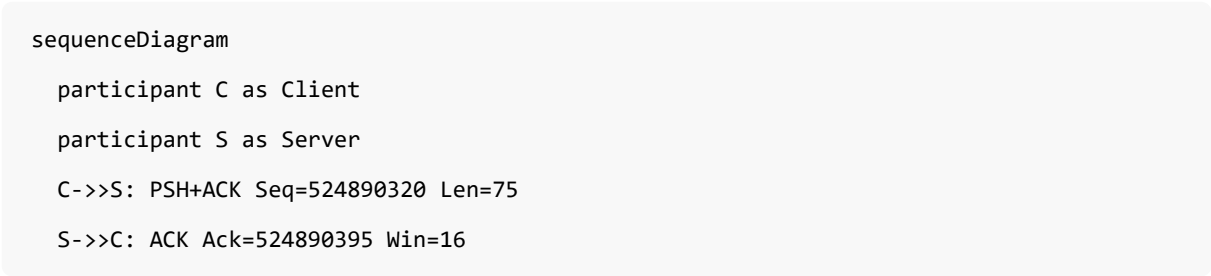


图 4 TCP 数据传输与确认示意

## 四次挥手

连接释放（帧 9 - 11，标准 FIN 关闭，无 RST）：

| 步骤 | 帧  | 方向            | Flags    | Seq        | Ack        | 说明                |
|----|----|---------------|----------|------------|------------|-------------------|
| 1  | 9  | Client→Server | FIN, ACK | 524890395  | 4130145356 | 客户端请求关闭           |
| 2  | 10 | Server→Client | ACK      | —          | 524890396  | 确认 FIN, Ack=u+1 ✓ |
| 3  | 10 | Server→Client | FIN      | 4130145356 | 524890396  | 服务器 FIN（同帧合并）     |

| 步骤 | 帧  | 方向            | Flags | Seq       | Ack        | 说明                 |
|----|----|---------------|-------|-----------|------------|--------------------|
| 4  | 11 | Client→Server | ACK   | 524890396 | 4130145357 | 最终确认，<br>Ack=v+1 ✓ |

表 11 TCP 四次挥手（逻辑四步）

帧 10 将「对 FIN 的 ACK」与「服务器 FIN」合并发送，属常见优化；逻辑上仍对应四次挥手中间两步。

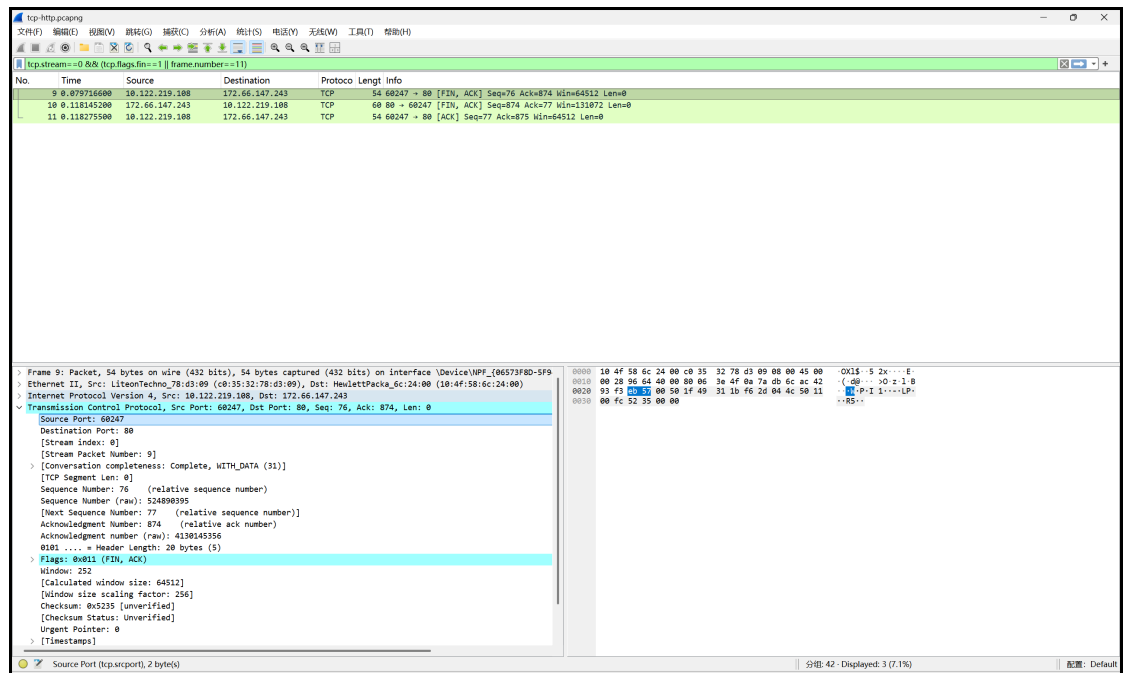


图 8 TCP 四次挥手 Wireshark 截图



图 5 TCP 四次挥手时序图

## 实验中遇到的问题和解决方案

---

### 1. DHCP 先出现 Release 包

现象: `ipconfig /release` 产生 DHCP Release (帧 1), 与后续 DORA 的 Transaction ID 不同。

解决: 分析时按 Transaction ID `0x348015F3` 筛选帧 2 - 5, 忽略 Release 帧。

### 2. ARP 由网关主动询问

现象: Request 由网关 `10.122.192.1` 发出, 非本机广播询问网关。

解决: 仍为一组有效 Opcode 1/2 配对, 重点分析字段与单播回复机制。

### 3. TCP 改用 HTTP 80 端口

现象: HTTPS 连接常以 RST 快速关闭, 不符合四次挥手分析。

解决: `curl -4 http://example.com`, 保存 `tcp-http.pcapng`, 使用 `tcp.stream==0` 分析。

## 实验心得

---

通过本次实验，我将 TCP/IP 协议栈与真实流量一一对应：Wireshark 三栏让抽象首部变成可验证的数据；手工计算 IP 校验和（`0x6820`）加深了对反码求和的理解；IP 分片表直观展示了 MTU 限制；DHCP 四步广播/单播差异、ARP 地址解析、TCP 序号 +1 规律，都在抓包中得到印证。今后排查网络问题，我会先用显示过滤器定位，再逐层展开协议树分析。



本报告由 `report/lab2-report.typ` 编译生成。源码整合自 `notes/` 下全部分析笔记与 `diagrams/` 时序图。

编译命令: `typst compile report/lab2-report.typ report/lab2-report.pdf`